

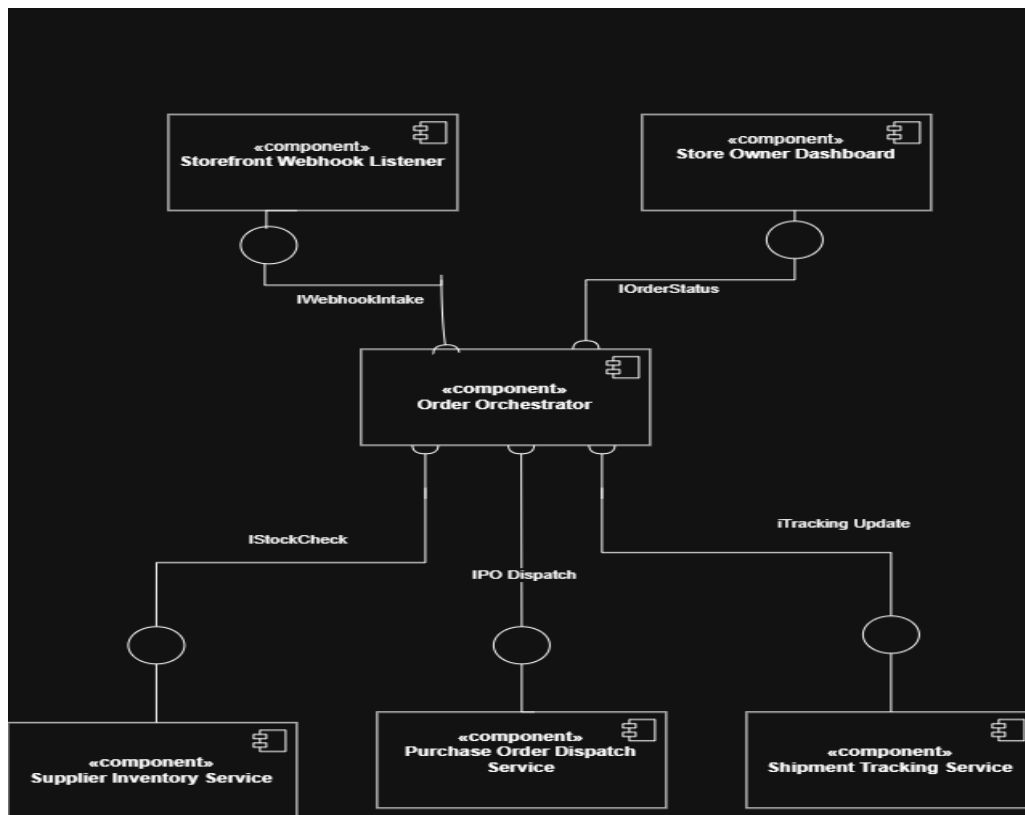
Lab 3: Component Modelling & Architectural Pattern Selection

Name: Shravani Prasad Joshi

SRN: PES1UG24AM264

Scenario No: 38 Dropshipping Inventory & Order Orchestrator

Component Diagram:



Architecture Selection

We chose Microservices Architecture for the Dropshipping Inventory & Order Orchestrator system.

Two Scenario-Related Reasons

- Independent scaling for the ingestion path: NFR-001 requires inbound Shopify/WooCommerce webhooks to be acknowledged within 200 ms to avoid provider retry storms. As a standalone microservice, the Storefront Webhook Listener can be scaled and tuned purely for low-latency ingestion, independent of the slower, I/O-bound work done downstream (supplier stock checks, PO dispatch, tracking).
- Fault isolation across multi-vendor order splitting: because a single storefront order can fan out into several supplier purchase orders, a slow or failing third-party Supplier API must not stall the whole order pipeline. Microservices let the Supplier Inventory Service and Purchase

Order Dispatch Service fail or degrade independently (with retries/circuit breakers) while the Order Orchestrator keeps processing other vendors' items.

Security Advantage

Each service can be issued its own credentials and network access boundary — for example, only the Supplier Inventory Service and Purchase Order Dispatch Service hold outbound supplier-API keys. If a supplier integration is compromised, the blast radius is limited to that service; the webhook ingestion path, order data, and store-owner dashboard are not directly exposed, satisfying the security half of NFR-001.

Performance Benefit

Decoupling ingestion from fulfilment lets the Storefront Webhook Listener immediately persist and acknowledge (HTTP 200) each webhook, then hand off processing asynchronously to the Order Orchestrator. This keeps webhook response time low and predictable even during peak load, without waiting on slower supplier-network round trips.